

HANDS-ON LAB GUIDE

FastAPI RAG Query System with SSE Streaming & Redis Caching

FastAPI

Server-Sent
Events

Redis

OpenAI

FAISS

LangChain

Platform	Duration	Level	Prerequisites
AWS EC2 Ubuntu + Code-Server	3–4 Hours	Intermediate	Python, REST APIs, Linux basics

Enterprise AI Training Program • AWS EC2 • Code-Server • OpenAI GPT-4

TABLE OF CONTENTS

Module 0	Lab Overview & Architecture
Module 1	Environment Setup on EC2
Module 2	Project Structure & Dependencies
Module 3	Build the RAG Pipeline (FAISS + OpenAI)
Module 4	FastAPI RAG Query Endpoint
Module 5	SSE Streaming with FastAPI
Module 6	Redis Caching Layer
Module 7	Integrating All Three Components
Module 8	Testing with curl & Python Client
Module 9	Troubleshooting & FAQs

MODULE 0 — Lab Overview & Architecture

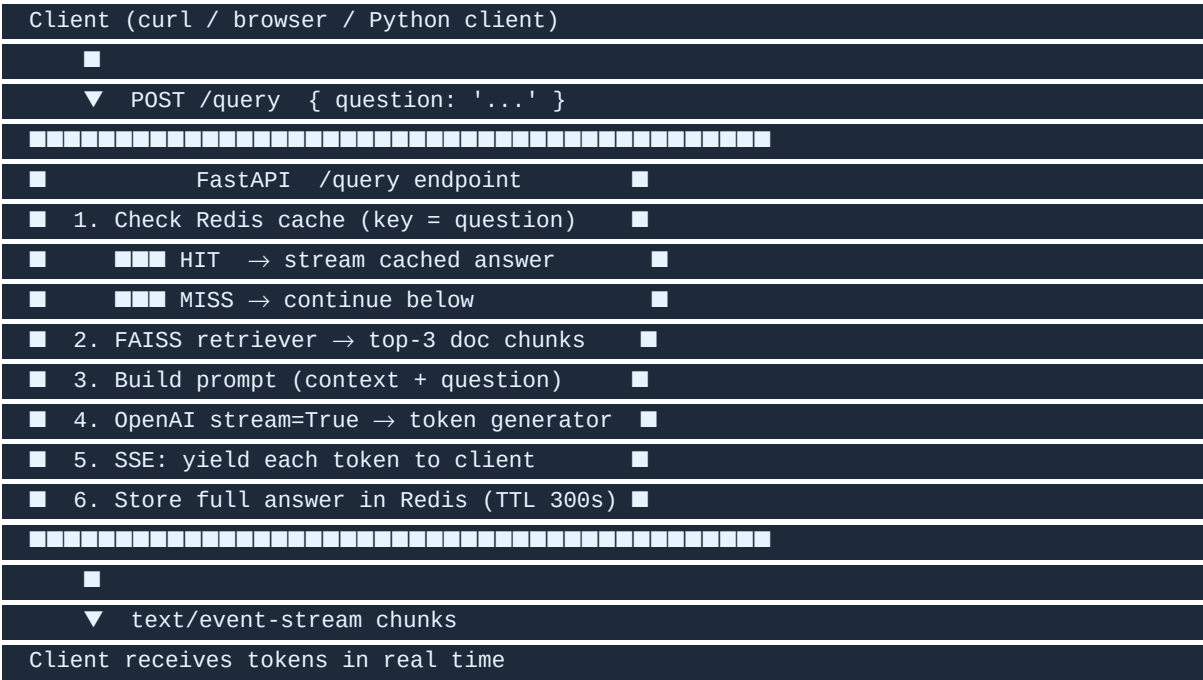
What You Will Build

In this lab you will build a production-style **Retrieval-Augmented Generation (RAG)** API service using FastAPI. The service accepts a natural-language question, retrieves relevant context from a local FAISS vector store, and streams the LLM answer back to the client using **Server-Sent Events (SSE)**. A **Redis** caching layer intercepts repeated queries so that identical questions are answered instantly from cache — eliminating redundant OpenAI API calls and reducing cost.

System Architecture Overview

Layer	Technology	Role
API Gateway	FastAPI (Uvicorn)	Receives HTTP requests, routes to handlers
Cache	Redis (localhost:6379)	Caches question→answer pairs with TTL
Retriever	FAISS + LangChain	Finds top-k relevant document chunks
LLM	OpenAI GPT-4o-mini	Generates answer with streaming enabled
Streaming	SSE / EventSourceResponse	Pushes token chunks to client in real time

Request Flow Diagram (text representation)



MODULE 1 — Environment Setup on EC2

1.1 Open Code-Server Terminal

Launch your Code-Server in the browser (<http://<EC2-PUBLIC-IP>:8080>). Open a new terminal via **Terminal** → **New Terminal**.

Tip: Ensure your EC2 Security Group allows inbound TCP on ports 8080 (Code-Server), 8000 (FastAPI), and 6379 (Redis).

1.2 Update System Packages

```
# Update package index
sudo apt-get update -y

# Upgrade existing packages
sudo apt-get upgrade -y
```

1.3 Install Redis Server

Redis will run as a local in-memory cache on the same EC2 instance.

```
# Install Redis
sudo apt-get install redis-server -y

# Start Redis service
sudo systemctl start redis-server

# Enable Redis to start on boot
sudo systemctl enable redis-server

# Verify Redis is running
redis-cli ping

# Expected output: PONG

Expected: Running redis-cli ping should return PONG. If it shows an error, run: sudo systemctl status redis-server
```

1.4 Install Python 3.11 & pip

```
# Check existing Python version
python3 --version

# Install pip if missing
sudo apt-get install python3-pip -y

# Install venv module
```

```
sudo apt-get install python3-venv -y
```

1.5 Create & Activate Virtual Environment

```
# Go to home directory
```

```
cd ~
```

```
# Create project folder
```

```
mkdir rag_api && cd rag_api
```

```
# Create virtual environment
```

```
python3 -m venv venv
```

```
# Activate it
```

```
source venv/bin/activate
```

```
# Verify (should show venv python)
```

```
which python
```

```
# Expected: /home/ubuntu/rag_api/venv/bin/python
```

Note: Always activate the virtualenv before running any Python commands in this lab: **source**
~/rag_api/venv/bin/activate

MODULE 2 — Project Structure & Dependencies

2.1 Install Python Dependencies

```
# Install all required packages
pip install fastapi uvicorn[standard] openai langchain
pip install langchain-openai langchain-community
pip install faiss-cpu tiktoken redis sse-starlette
pip install python-dotenv sentence-transformers
```

Package Reference

Package	Version	Purpose
fastapi	>=0.109	Web framework for building APIs
uvicorn[standard]	>=0.27	ASGI server to run FastAPI
openai	>=1.12	OpenAI Python SDK (streaming)
langchain	>=0.1	RAG orchestration framework
langchain-openai	>=0.0.8	LangChain OpenAI embeddings/chat
faiss-cpu	>=1.7	Vector similarity search index
redis	>=5.0	Redis Python client
sse-starlette	>=1.6	Server-Sent Events for Starlette/FastAPI
python-dotenv	>=1.0	Load .env file variables
tiktoken	>=0.6	Token counter for OpenAI models

2.2 Project Directory Structure

```
rag_api/
├── venv/                # virtual environment
├── .env                 # API keys (never commit!)
├── main.py              # FastAPI app entry point
├── rag_pipeline.py       # RAG logic: FAISS + OpenAI
├── cache.py             # Redis cache helpers
├── streaming.py          # SSE streaming helpers
├── sample_docs/
│   ├── knowledge_base.txt # Sample knowledge base
│   └── requirements.txt   # Dependency list
```

2.3 Create .env File (API Keys)

```
# Create .env file in project root
```

```
nano .env  
  
  
# Add the following content:  
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx  
REDIS_HOST=localhost  
REDIS_PORT=6379  
REDIS_TTL=300  
FAISS_INDEX_PATH=./faiss_index
```

Security Warning: Never commit your .env file to Git. Add .env to .gitignore immediately.

2.4 Create requirements.txt

```
# Save dependencies to file  
pip freeze > requirements.txt  
  
# Later, restore environment with:  
# pip install -r requirements.txt
```

MODULE 3 — Build the RAG Pipeline (FAISS + OpenAI)

The RAG pipeline has two phases: **Indexing** (offline) and **Retrieval** (online). During indexing, documents are chunked, embedded using OpenAI's embedding model, and stored in a FAISS index on disk. During retrieval, the user's question is embedded and the most similar chunks are fetched to form the LLM context.

3.1 Create Sample Knowledge Base

```
# Create sample_docs directory
mkdir -p sample_docs

# Create knowledge_base.txt
cat > sample_docs/knowledge_base.txt << 'EOF'
FastAPI is a modern Python web framework for building APIs.
It is based on standard Python type hints and supports async natively.
FastAPI automatically generates OpenAPI documentation.

Redis is an in-memory key-value store used for caching.
Redis supports data structures like strings, hashes, lists, and sets.
Redis TTL (Time-To-Live) allows automatic expiry of cached keys.

Retrieval-Augmented Generation (RAG) combines retrieval with generation.
RAG systems first retrieve relevant documents then generate answers.
FAISS (Facebook AI Similarity Search) is used for fast vector retrieval.

Server-Sent Events (SSE) allow servers to push data to clients.
SSE uses a persistent HTTP connection and text/event-stream content type.
SSE is simpler than WebSockets for one-way server-to-client streaming.
EOF
```

3.2 Create rag_pipeline.py

This module handles document loading, FAISS indexing, and retrieval.

```
# File: rag_pipeline.py
import os
from dotenv import load_dotenv
from langchain_community.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

load_dotenv()

OPENAI_API_KEY = os.getenv('OPENAI_API_KEY')
```



```
FAISS_INDEX_PATH = os.getenv('FAISS_INDEX_PATH', './faiss_index')

embeddings = OpenAIEmbeddings(
    openai_api_key=OPENAI_API_KEY,
    model='text-embedding-3-small'
)

def build_faiss_index(doc_path: str = 'sample_docs/knowledge_base.txt'):
    '''Load documents, split into chunks, build FAISS index.'''
    # Step 1: Load raw text
    loader = TextLoader(doc_path)
    documents = loader.load()
    print(f'Loaded {len(documents)} document(s)')

    # Step 2: Split into overlapping chunks
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=300,
        chunk_overlap=50,
        separators=['\n\n', '\n', '.', ' ']
    )
    chunks = splitter.split_documents(documents)
    print(f'Created {len(chunks)} chunks')

    # Step 3: Create embeddings and FAISS index
    vector_store = FAISS.from_documents(chunks, embeddings)
    vector_store.save_local(FAISS_INDEX_PATH)
    print(f'FAISS index saved to {FAISS_INDEX_PATH}')
    return vector_store

def load_faiss_index():
    '''Load existing FAISS index from disk.'''
    if not os.path.exists(FAISS_INDEX_PATH):
        print('FAISS index not found. Building...')
        return build_faiss_index()
    return FAISS.load_local(
        FAISS_INDEX_PATH,
        embeddings,
        allow_dangerous_deserialization=True
    )

def retrieve_context(question: str, k: int = 3) -> str:
    '''Retrieve top-k relevant chunks for a question.'''
    vector_store = load_faiss_index()
    docs = vector_store.similarity_search(question, k=k)
    context = '\n\n'.join([doc.page_content for doc in docs])
```

```
        return context

# --- Test ---
if __name__ == '__main__':
    build_faiss_index()
    ctx = retrieve_context('What is FastAPI?')
    print('Retrieved Context:\n', ctx)
```

3.3 Run the Indexing Script

```
# Build FAISS index (run once)
python rag_pipeline.py

# Expected output:
# Loaded 1 document(s)
# Created 8 chunks
# FAISS index saved to ./faiss_index
# Retrieved Context:
# FastAPI is a modern Python web framework...
```

Checkpoint: You should see a **faiss_index/** folder created with **index.faiss** and **index.pkl** files.

MODULE 4 — FastAPI RAG Query Endpoint

Now we build the FastAPI application with the RAG query endpoint. The endpoint accepts a POST request with a JSON body containing the question, retrieves context from FAISS, and returns the LLM answer.

4.1 Create main.py — Basic FastAPI App

```
# File: main.py
import os
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from dotenv import load_dotenv
from openai import OpenAI
from rag_pipeline import retrieve_context

load_dotenv()

app = FastAPI(
    title='RAG Query API',
    description='FastAPI + FAISS + OpenAI RAG with SSE Streaming and Redis Cache',
    version='1.0.0'
)

# Allow all origins for lab purposes
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_methods=['*'],
    allow_headers=['*'],
)

client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

# --- Request/Response Models ---
class QueryRequest(BaseModel):
    question: str
    max_tokens: int = 512
    temperature: float = 0.3

class QueryResponse(BaseModel):
    question: str
    answer: str
    source: str # 'cache' or 'llm'
    context_used: str
```

```

# --- Health Check ---
@app.get('/health')
def health_check():
    return {'status': 'ok', 'service': 'RAG API'}

# --- Basic (non-streaming) RAG endpoint ---
@app.post('/query', response_model=QueryResponse)
def query_rag(request: QueryRequest):
    try:
        context = retrieve_context(request.question)
        prompt = f'''You are a helpful assistant.
Use ONLY the following context to answer the question.
If unsure, say 'I don't have enough information.'

Context:
{context}

Question: {request.question}
Answer:'''

        response = client.chat.completions.create(
            model='gpt-4o-mini',
            messages=[{'role': 'user', 'content': prompt}],
            max_tokens=request.max_tokens,
            temperature=request.temperature
        )
        answer = response.choices[0].message.content
        return QueryResponse(
            question=request.question,
            answer=answer,
            source='llm',
            context_used=context
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

4.2 Run the FastAPI Server

```

# Run with uvicorn (auto-reload for development)
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# Expected output:
# INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
# INFO:      Started reloader process

```

4.3 Test the Basic Endpoint

```
# Open a NEW terminal tab in Code-Server and test:
```

```
# Health check
```

```
curl http://localhost:8000/health
```

```
# RAG query
```

```
curl -X POST http://localhost:8000/query \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"question": "What is FastAPI?"}'
```

```
# View auto-generated API docs
```

```
# Open in browser: http://<EC2-PUBLIC-IP>:8000/docs
```

Auto Docs: FastAPI generates interactive Swagger UI at **/docs** and ReDoc at **/redoc** automatically — no extra config needed.

MODULE 5 — SSE Streaming Response from LLM

Server-Sent Events (SSE) enable the server to push data to the client over a single long-lived HTTP connection. Instead of waiting for the full LLM response, the client receives each token as it is generated — creating a ChatGPT-like streaming experience.

How SSE Works

```
# SSE Protocol – each event looks like:
```

```
data: {"token": "FastAPI"}
```

```
data: {"token": " is"}
```

```
data: {"token": " a"}
```

```
data: [DONE]
```

```
# Content-Type: text/event-stream
```

```
# Connection: keep-alive
```

5.1 Create streaming.py

```
# File: streaming.py
```

```
import json
```

```
import os
```

```
from typing import AsyncGenerator
```

```
from openai import AsyncOpenAI
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
async_client = AsyncOpenAI(api_key=os.getenv('OPENAI_API_KEY'))
```

```
async def stream_llm_response(
```

```
    prompt: str,
```

```
    model: str = 'gpt-4o-mini',
```

```
    max_tokens: int = 512,
```

```
    temperature: float = 0.3
```

```
) -> AsyncGenerator[str, None]:
```

```
    '''Stream tokens from OpenAI as SSE events.'''
```

```
    full_answer = ''
```

```
    async with async_client.chat.completions.stream(
```

```
        model=model,
```

```
        messages=[{'role': 'user', 'content': prompt}],
```

```
        max_tokens=max_tokens,
```

```

        temperature=temperature
    ) as stream:
        async for text in stream.text_stream:
            full_answer += text
            # Yield each token as SSE-formatted JSON
            yield f"data: {json.dumps({'token': text})}\n\n"

        # Signal end of stream
        yield f"data: {json.dumps({'token': '', 'done': True, 'full_answer': full_answer})}\n\n"
        yield 'data: [DONE]\n\n'

```

5.2 Add SSE Streaming Endpoint to main.py

Add the following imports and endpoint to your existing **main.py**:

```

# Add these imports at the top of main.py
import json
from sse_starlette.sse import EventSourceResponse
from streaming import stream_llm_response

# --- SSE Streaming RAG Endpoint ---
@app.post('/query/stream')
async def query_rag_stream(request: QueryRequest):
    """
    Streaming RAG endpoint using Server-Sent Events.
    Returns tokens as they are generated by the LLM.
    """
    # Retrieve context from FAISS
    context = retrieve_context(request.question)

    # Build the RAG prompt
    prompt = f'''You are a helpful assistant.
    Use ONLY the following context to answer the question.

    Context:
    {context}

    Question: {request.question}
    Answer:'''

    # Return SSE stream
    return EventSourceResponse(
        stream_llm_response(
            prompt=prompt,

```

```

        max_tokens=request.max_tokens,
        temperature=request.temperature
    ),
    media_type='text/event-stream'
)

```

5.3 Test SSE Streaming Endpoint

```

# Test streaming endpoint with curl
# The --no-buffer flag shows tokens as they arrive
curl -X POST http://localhost:8000/query/stream \
-H "Content-Type: application/json" \
-d '{"question": "Explain Redis caching"}' \
--no-buffer

# Expected output (tokens arrive one by one):
data: {"token": "Redis"}
data: {"token": " is"}
data: {"token": " an"}
data: {"token": " in-memory"}
# ... more tokens ...
data: {"token": "", "done": true, "full_answer": "Redis is an..."}
data: [DONE]

```

5.4 Python Client for SSE

Create a test client to consume the SSE stream:

```

# File: test_stream_client.py
import requests
import json

def stream_query(question: str):
    url = 'http://localhost:8000/query/stream'
    payload = {'question': question}
    headers = {'Content-Type': 'application/json'}

    with requests.post(url, json=payload,
                       headers=headers, stream=True) as resp:
        print(f'Streaming answer for: {question}')
        print('-' * 50)
        for line in resp.iter_lines():
            if line:
                decoded = line.decode('utf-8')
                if decoded.startswith('data: '):
                    data = decoded[6:]

```



```
        if data == '[DONE]':
            print('\n[Stream complete]')
            break
        try:
            obj = json.loads(data)
            if not obj.get('done'):
                print(obj['token'], end='', flush=True)
        except json.JSONDecodeError:
            pass

if __name__ == '__main__':
    stream_query('What is FAISS and how does it work?')
# Run the client
python test_stream_client.py
```

MODULE 6 — Redis Caching Layer

Redis caching prevents redundant OpenAI API calls for repeated questions. When the same question is asked, the answer is served from Redis in milliseconds instead of waiting for the LLM to generate a response — saving both time and API cost.

6.1 Caching Strategy

Scenario	Behaviour	Latency
Cache MISS (first request)	Retrieve → LLM → Store in Redis	2–5 seconds
Cache HIT (repeat request)	Return from Redis instantly	< 5 ms
Cache EXPIRED (after TTL)	Treated as MISS, re-queries LLM	2–5 seconds

6.2 Create cache.py

```
# File: cache.py
import os
import redis
import hashlib
from dotenv import load_dotenv

load_dotenv()

REDIS_HOST = os.getenv('REDIS_HOST', 'localhost')
REDIS_PORT = int(os.getenv('REDIS_PORT', 6379))
REDIS_TTL = int(os.getenv('REDIS_TTL', 300)) # 5 minutes

# Create Redis client
redis_client = redis.Redis(
    host=REDIS_HOST,
    port=REDIS_PORT,
    db=0,
    decode_responses=True # Return str instead of bytes
)

def make_cache_key(question: str) -> str:
    """
    Create a consistent cache key from a question.
    Normalize: lowercase, strip whitespace, then MD5 hash.
    """
    normalized = question.lower().strip()
    return 'rag:' + hashlib.md5(normalized.encode()).hexdigest()
```

```

def get_cached_answer(question: str) -> str | None:
    '''Return cached answer or None if not found.'''
    key = make_cache_key(question)
    return redis_client.get(key)

def set_cached_answer(question: str, answer: str) -> None:
    '''Store answer in Redis with TTL.'''
    key = make_cache_key(question)
    redis_client.setex(key, REDIS_TTL, answer)

def delete_cached_answer(question: str) -> bool:
    '''Manually invalidate a cached answer.'''
    key = make_cache_key(question)
    return bool(redis_client.delete(key))

def get_cache_stats() -> dict:
    '''Return Redis server statistics.'''
    info = redis_client.info()
    return {
        'connected_clients': info['connected_clients'],
        'used_memory_human': info['used_memory_human'],
        'keyspace_hits': info['keyspace_hits'],
        'keyspace_misses': info['keyspace_misses'],
    }

# --- Test ---
if __name__ == '__main__':
    set_cached_answer('test question', 'test answer')
    print(get_cached_answer('test question'))
    print(get_cache_stats())

```

6.3 Test Cache Module Independently

```

# Test the cache module
python cache.py

# Expected output:
# test answer
# {'connected_clients': 1, 'used_memory_human': '...',
#  'keyspace_hits': 1, 'keyspace_misses': 0}

# Inspect keys in Redis directly:
redis-cli keys 'rag:*'
redis-cli ttl <key>          # Check remaining TTL

```

MODULE 7 — Integrating All Three Components

Now we integrate the RAG pipeline, SSE streaming, and Redis caching into the final unified **main.py**. The complete endpoint checks the cache first, and if no cached answer exists, it retrieves context, streams from the LLM, and caches the final answer.

7.1 Final Complete main.py

[illegible]


```

    )
    answer = response.choices[0].message.content

    # 4. Store in Redis cache
    set_cached_answer(req.question, answer)

    return QueryResponse(
        question=req.question,
        answer=answer,
        source='llm',
        context_used=context
    )

# --- SSE Streaming with cache-aware logic ---
@app.post('/query/stream')
async def query_rag_stream(req: QueryRequest):
    context = retrieve_context(req.question)
    prompt = build_rag_prompt(req.question, context)

    async def event_generator() -> AsyncGenerator[str, None]:
        # Check cache first - stream from cache if hit
        cached = get_cached_answer(req.question)
        if cached:
            # Simulate streaming from cache for consistent UX
            words = cached.split(' ')
            for word in words:
                token = word + ' '
                yield f"data: {json.dumps({'token': token, 'source': 'cache'})}\n\n"
            yield f"data: {json.dumps({'done': True, 'source': 'cache'})}\n\n"
            yield 'data: [DONE]\n\n'
            return

        # Cache MISS: stream from LLM
        full_answer = ''
        async with async_client.chat.completions.stream(
            model='gpt-4o-mini',
            messages=[{'role': 'user', 'content': prompt}],
            max_tokens=req.max_tokens,
            temperature=req.temperature
        ) as stream:
            async for text in stream.text_stream:
                full_answer += text
                yield f"data: {json.dumps({'token': text, 'source': 'llm'})}\n\n"

        # Store in cache after streaming is complete

```

```
set_cached_answer(req.question, full_answer)
```

```
yield f"data: {json.dumps({'done': True, 'source': 'llm', 'cached': True})}\n\n"
```

```
yield 'data: [DONE]\n\n'
```

```
return EventSourceResponse(event_generator(),
```

```
media_type='text/event-stream')
```

7.2 Restart the Server

```
# Stop current server (Ctrl+C) then restart
```

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```



```
print('='*55)
start = time.time()
resp = requests.post(f'{BASE_URL}/query',
                    json={'question': question})
elapsed = round(time.time() - start, 3)
data = resp.json()
print(f'Source : {data["source"]}')
print(f'Latency : {elapsed}s')
print(f'Answer : {data["answer"][:120]}...')
return data

def test_stream(question):
    print(f'\nStreaming: {question}')
    print('='*55)
    with requests.post(f'{BASE_URL}/query/stream',
                    json={'question': question,
                        stream=True}) as r:
        for line in r.iter_lines():
            if line:
                decoded = line.decode('utf-8')
                if decoded.startswith('data: '):
                    data_str = decoded[6:]
                    if data_str == '[DONE]':
                        print('\n[Done]')
                        break
                    try:
                        obj = json.loads(data_str)
                        if not obj.get('done'):
                            print(obj['token'], end='', flush=True)
                    except Exception:
                        pass

if __name__ == '__main__':
    q = 'What is Server-Sent Events?'
    # 1st call: should hit LLM
    r1 = test_query(q, 'CALL 1 – expect LLM')
    # 2nd call: should hit cache
    r2 = test_query(q, 'CALL 2 – expect Redis cache')
    assert r1['answer'] == r2['answer'], 'Cache mismatch!'
    print('\nCache consistency: PASSED')
    # Streaming test
    test_stream('How does RAG work?')
# Run full test
python test_all.py
```

MODULE 9 — Troubleshooting & FAQs

9.1 Common Errors & Fixes

Error: `redis.exceptions.ConnectionError`

Cause: Redis server not running

Fix:

```
sudo systemctl start redis-server && redis-cli ping
```

Error: `openai.AuthenticationError`

Cause: Invalid or missing API key in `.env`

Fix:

```
cat .env # verify key starts with sk- | source .env
```

Error: `ModuleNotFoundError: No module named 'sse_starlette'`

Cause: Package not installed in virtualenv

Fix:

```
source venv/bin/activate && pip install sse-starlette
```

Error: `FAISS index not found`

Cause: Index never built or wrong path in `.env`

Fix:

```
python rag_pipeline.py # rebuilds the index
```

Error: `OSError: [Errno 98] Address already in use`

Cause: Port 8000 already occupied

Fix:

```
lsof -i :8000 | kill -9 <PID> then restart
```

Error: `curl: (7) Failed to connect`

Cause: Security Group missing inbound rule for port 8000

Fix:

```
AWS Console → EC2 → Security Groups → add inbound TCP 8000
```

Error: streaming stops after first token

Cause: Missing `--no-buffer` in curl or sync client

Fix:

```
Add --no-buffer to curl; use stream=True in requests
```

Error: Cache always MISS

Cause: Questions not normalized before hashing

Fix:

```
Check make_cache_key() strips + lowercases input
```

9.2 Useful Diagnostic Commands

```
# Check Redis connectivity
redis-cli ping && redis-cli info server | grep version

# List all RAG cache keys
redis-cli keys 'rag:*'

# Flush all cache (use carefully!)
redis-cli flushdb

# Check FastAPI logs
# (they appear in the terminal where uvicorn is running)

# Check running processes on port 8000
lsof -i :8000

# Check virtual environment is active
which python # should show venv/bin/python

# View installed packages
pip list | grep -E 'fastapi|openai|redis|faiss|sse'

# Test OpenAI key directly
curl https://api.openai.com/v1/models \
  -H "Authorization: Bearer $OPENAI_API_KEY" | head -c 200
```

9.3 Quick Reference Card

Action	Command
Start FastAPI server	uvicorn main:app --host 0.0.0.0 --port 8000 --reload
Activate virtualenv	source ~/rag_api/venv/bin/activate
Build FAISS index	python rag_pipeline.py
Ping Redis	redis-cli ping
View Swagger UI	http://:8000/docs

Test non-stream query	<code>curl -X POST localhost:8000/query -H 'Content-Type: application/json' -d '{"question": "..."}'</code>
Test SSE stream	<code>curl -X POST localhost:8000/query/stream ... --no-buffer</code>
Flush Redis cache	<code>redis-cli flushdb</code>
Cache stats API	<code>curl http://localhost:8000/cache/stats</code>

Lab Complete: Congratulations! You have built a production-style RAG API with FastAPI, SSE token streaming, and Redis caching. The same pattern scales to larger document collections, different embedding models, and cloud-hosted Redis (ElastiCache).